

System Design Document

Sistema per la gestione dell'identità digitale
degli studenti delle istituzioni AFAM

Insegnamento di Ingegneria del Software / Progettazione del Software

Anno Accademico 2025 – 2026

Documento di Analisi dei Requisiti

Gruppo:
Chiarelli Antonino Maria
Riccardo Centonze
Federico Meli
Gabriele Mattia Guccione

Indice.....	1
-------------	---

1. Obiettivo del sistema	2
2. Architettura software attuale	3
3. Obiettivi di progettazione	3
4. Architettura software proposta.....	3
4.1 Panoramica	4
4.2 Requisiti minimi per l'utilizzo del software proposto	4
4.3 Scomposizione in sottosistemi.....	4
4.4 Mappatura hardware/software	5
5. Gestione dei dati persistenti	5
5.1 Schema E-R.....	6
5.2 Modello relazionale	6
5.3 Struttura delle tabelle	7
5.3.1 utente_afam	7
5.3.2 token	8
5.3.3 profilo	8
5.3.4 profilo_competenze	9
5.3.5 profilo_interessi	9
5.3.6 contenuto	10
5.3.7 contenuto_autore	10
5.3.8 contenuto_collaboratore.....	10
5.3.9 allegato	11
5.3.10 gruppo.....	11
5.3.11 link	11
5.3.12 gruppo_contenuto	12
6. Controllo degli accessi e sicurezza	12
7. Condizioni limite.....	13

1. Obiettivo del sistema

Il sistema ha l'obiettivo di fornire alle istituzioni AFAM (Alta Formazione Artistica, Musicale e Coreutica) uno strumento digitale che consenta agli studenti di rappresentare in modo efficace il proprio percorso artistico e formativo, costruendo una propria identità digitale unica. Tale identità include i dati anagrafici e accademici e la produzione artistica dello studente (registrazioni audio e video, spartiti, curriculum e altri materiali), organizzata in maniera flessibile e senza inviare multiple mail per visionare i contenuti di uno studente da parte di un visualizzatore. Il sistema permette allo studente di decidere quali parti del proprio profilo rendere visibili e in quali contesti, di condividerle anche con soggetti esterni mediante appositi collegamenti, e di verificare se i contenuti condivisi siano stati effettivamente visualizzati. L'accesso avviene mediante credenziali personali in modo sicuro, con funzionalità di gestione dell'account e di recupero delle credenziali.

2. Architettura software attuale

Allo stato attuale non esiste un sistema software unitario adottato dall'istituzione AFAM per la gestione dell'identità digitale degli studenti. La preparazione di portfolio, curriculum e raccolte di materiali multimediali avviene in modo manuale, tramite documenti separati, file sparsi o materiali inviati via e-mail e soprattutto senza alcun controllo dell'effettiva visualizzazione dei contenuti condivisi. Le attività che il sistema proposto intende automatizzare sono quindi attualmente svolte in maniera manuale e frammentata.

3. Obiettivi di progettazione

Gli obiettivi di progettazione (design goals) derivano dai requisiti non funzionali del RAD.

- **Sicurezza:** le password sono gestite secondo gli standard ACN mediante l'algoritmo di hashing Argon2 e devono contenere almeno 12 caratteri; l'email deve rispettare l'espressione regolare di validazione (regex); se attiva, l'autenticazione è rafforzata da un secondo fattore (OTP).
- **Validazione degli input:** il sistema sanifica e valida tutti i dati in ingresso, rifiutando i caratteri riservati indicati nel RAD (eccetto un singolo «@» nelle email).
- **Affidabilità e disponibilità:** il sistema gestisce le condizioni di errore, in particolare la perdita di connessione e l'assenza di risposta del DBMS o Server, riportando l'utente a uno stato valido e fornendo un riscontro chiaro mediante messaggi dedicati.
- **Usabilità:** l'interfaccia è chiara, intuitiva e orientata alla facilità d'uso, in considerazione di un'utenza non necessariamente esperta in ambito informatico (User Friendly).
- **Considerazioni legislative:** il trattamento dei dati personali e la tutela dei materiali artistici avvengono con adeguate garanzie di protezione, nel rispetto della normativa vigente.

4. Architettura software proposta

4.1 Panoramica

Il sistema adotta un'architettura client-server a tre livelli (three-tier). Il livello di presentazione è un client a singola pagina (Single Page Application) realizzato in Angular, eseguito nel browser dell'utente. Il livello applicativo è un'applicazione server Spring Boot che espone interfacce REST su HTTP/JSON e implementa la logica di funzionamento. Il livello dei dati è una base di dati relazionale PostgreSQL. La comunicazione fra i livelli è disaccoppiata: il client dialoga con il server unicamente tramite API REST, mentre il server accede al DBMS tramite uno strato di persistenza dedicato (DBMSBoundary), realizzato con repository Spring Data JPA. Internamente il server è organizzato a strati secondo il pattern Boundary-Control-Entity: gli oggetti control (RestController) gestiscono la logica, le classi entity modellano il dominio e lo strato DBMSBoundary realizza l'accesso ai dati, rispettando un paradigma di tipo repository in cui il DBMS centrale costituisce il punto di integrazione fra i sottosistemi. L'autenticazione è stateless e basata su token JWT.

4.2 Requisiti minimi per l'utilizzo del software proposto

Per il funzionamento del sistema sono necessari: lato client, un browser web moderno con JavaScript abilitato e una connessione ad Internet stabile; lato server, un ambiente di esecuzione containerizzato (Docker) in grado di ospitare l'applicazione Spring Boot (JVM) e il DBMS PostgreSQL; un servizio di posta elettronica (SMTP) per l'invio di OTP ed email di recupero credenziali. È inoltre richiesta una casella di posta elettronica istituzionale per la registrazione e la verifica dell'identità.

4.3 Scomposizione in sottosistemi

Il sistema è scomposto nei seguenti sottosistemi, ciascuno con responsabilità ben definite. La comunicazione fra i sottosistemi applicativi avviene attraverso il sottosistema di persistenza e, indirettamente, attraverso il DBMS, secondo il paradigma repository.

Sottosistema	Responsabilità
Client SPA (Angular)	Presentazione: raccoglie l'input dell'utente, mostra i dati, gestisce routing e protezione delle rotte; comunica con il server via REST.
Gestione Account	Registrazione, autenticazione (credenziali locali o Identity Provider), 2FA/OTP, logout, recupero credenziali. Control: AuthControl, RegistrationControl; Servizi JwtService, EmailService.
Gestione Profilo	Lettura e modifica di credenziali, password, anagrafica e altre informazioni del profilo; profilo pubblico. Control: ProfileControl.
Gestione Contenuti	Caricamento, consultazione, modifica, eliminazione e raggruppamento dei contenuti; gestione allegati e gruppi. Control: UploadControl, ContentControl, ViewControl.
Condivisione	Generazione, configurazione, revoca e fruizione pubblica dei link; verifica delle visualizzazioni. Control: ShareControl.
Ricerca	Ricerca di utenti, gruppi, contenuti, autori e collaboratori. Control: SearchControl.

Persistenza (DBMSBoundary)	Strato repository che incapsula l'accesso al DBMS tramite Spring Data JPA/Hibernate.
DBMS (PostgreSQL)	Memorizzazione e recupero dei dati persistenti; attore esterno passivo del sistema.

All'interno di ciascun sottosistema applicativo si ritrova la struttura BCE: un livello di interfaccia (boundary), un livello di controllo (control) che gestisce la logica, e un livello di accesso ai dati che comunica con il DB centrale, coerentemente con l'architettura repository adottata.

4.4 Mappatura hardware/software

Il sistema è distribuito su tre nodi logici, realizzati come container Docker orchestrati tramite docker-compose e collegati da una rete dedicata.

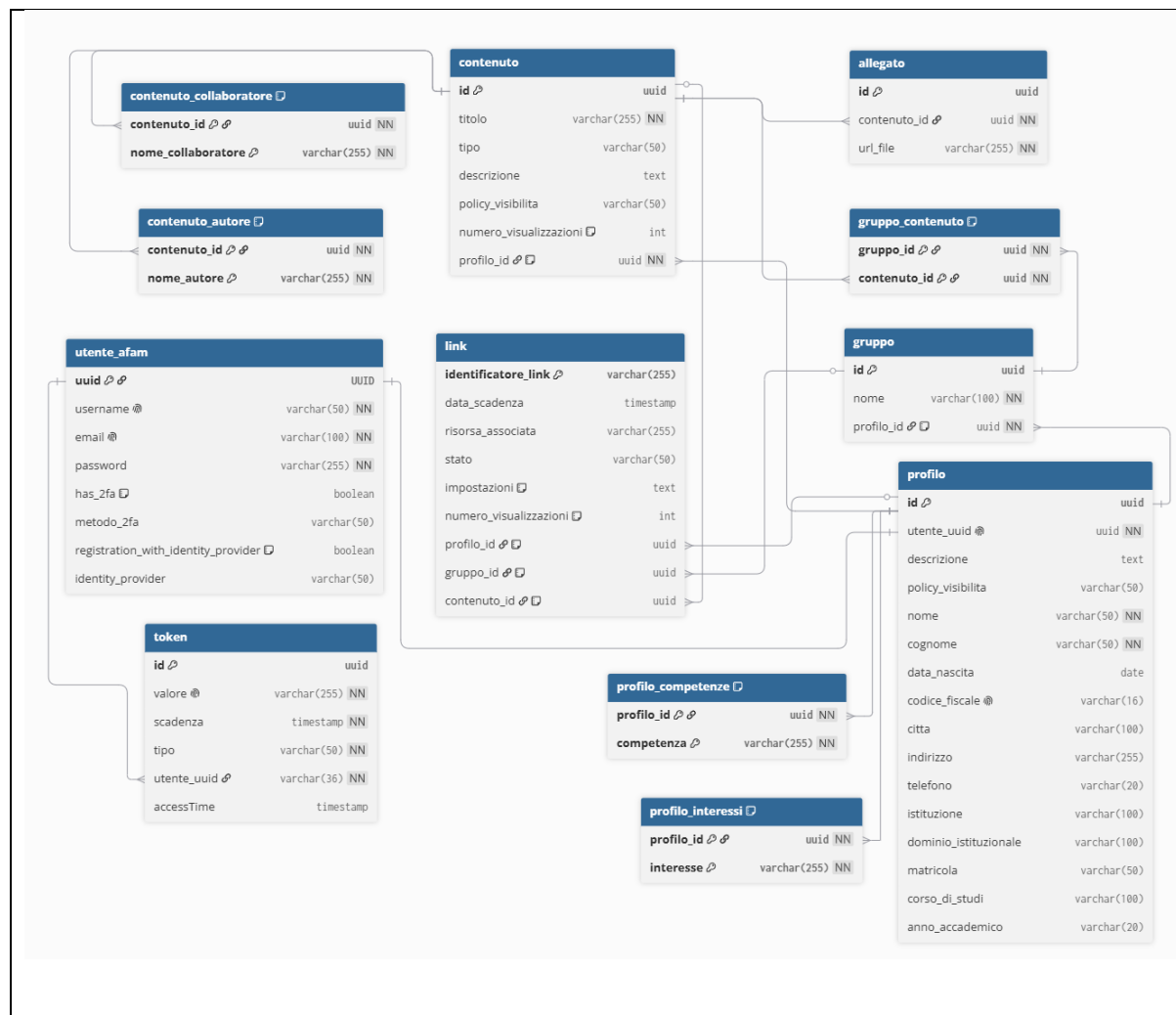
Nodo	Componente software
Nodo Frontend	Server web (es. Nginx) che serve l'applicazione Angular compilata; espone l'interfaccia all'utente.
Nodo Backend	Java Virtual Machine con l'applicazione Spring Boot (control, service, DBMSBoundary); espone le API REST sulla porta applicativa.
Nodo Database (DBHost)	Istanza PostgreSQL che memorizza i dati e li fornisce al backend tramite query.

Il client comunica con il nodo Backend esclusivamente via API REST; il nodo Backend comunica con il nodo Database tramite driver JDBC. La separazione su nodi distinti consente scalabilità, isolamento dei guasti e una più semplice manutenzione. I file caricati dagli utenti sono archiviati sul nodo Backend in una directory organizzata per UUID dell'utente proprietario, mentre i relativi metadati e percorsi sono persistiti nel DBMS.

5. Gestione dei dati persistenti

5.1 Schema E-R

Il modello concettuale individua l'entità centrale UtenteAfam (account) in relazione uno-a-uno con Profilo, che rappresenta l'identità dello studente. Il Profilo raccoglie i propri Contenuti (portfolio) e i propri Gruppi; ogni Contenuto può avere più Allegati e può appartenere a più Gruppi (relazione molti-a-molti). Profilo, Contenuto e Gruppo sono risorse condivisibili e possono essere associati a uno o più Link. L'account è inoltre associato a uno o più Token (registrazione, 2FA, recupero credenziali). Le associazioni multi-a-molti e gli attributi multivalore (autori, collaboratori, allegati) sono risolti con apposite tabelle.



5.2 Modello relazionale

Dalla traduzione dello schema E-R derivano le relazioni seguenti (le chiavi primarie sono sottolineate concettualmente; FK indica le chiavi esterne).

-) **utente_afam** (uid, username, email, password, has_2fa, metodo_2fa, registration_with_identity_provider, identity_provider) — PK: uid;

-) **token** (id, valore, scadenza, tipo, utente_uid, accessTime) — PK: id; FK: utente_uid → utente_afam(uid);

-)**profilo** (id, utente_uuid, descrizione, policy_visibilita, nome, cognome, data_nascita, codice_fiscale, citta, indirizzo, telefono, istituzione, dominio_istituzionale, matricola, corso_di_studi, anno_accademico) — PK: id; FK: utente_uuid → utente_afam(uuid);

-)**profilo_competenze** (profilo_id, competenza) — PK: (profilo_id, competenza); FK: profilo_id → profilo(id);

-)**profilo_interessi** (profilo_id, interesse) — PK: (profilo_id, interesse); FK: profilo_id → profilo(id);

-)**contenuto** (id, titolo, tipo, descrizione, policy_visibilita, numero_visualizzazioni, profilo_id) — PK: id; FK: profilo_id → profilo(id);

-)**contenuto_autore** (contenuto_id, nome_autore) — **PK: contenuto_id**; FK: contenuto_id → contenuto(id);

-)**contenuto_collaboratore** (contenuto_id, nome_collaboratore) — **PK: contenuto_id**; FK: contenuto_id → contenuto(id);

-)**allegato** (id, contenuto_id, url_file) — PK: id; FK: contenuto_id → contenuto(id);

-)**gruppo** (id, nome, profilo_id) — PK: id; FK: profilo_id → profilo(id);

-)**gruppo_contenuto** (gruppo_id, contenuto_id) — **PK: (gruppo_id, contenuto_id)**; FK: gruppo_id → gruppo(id), contenuto_id → contenuto(id);

-)**link** (identificatore_link, data_scadenza, risorsa_associata, stato, impostazioni, numero_visualizzazioni, profilo_id, gruppo_id, contenuto_id) — PK: identificatore_link; FK: profilo_id → profilo(id), gruppo_id → gruppo(id), contenuto_id → contenuto(id)

5.3 Struttura delle tabelle

Di seguito la struttura fisica delle principali tabelle, con tipo e vincoli degli attributi.

5.3.1 utente_afam

Attributo	Tipo	Vincoli	Descrizione
uuid	varchar(36)	PK	Identificativo univoco dell'account
username	varchar(50)	NOT NULL, UNIQUE	Nome utente, gestito dal sistema
email	varchar(100)	NOT NULL, UNIQUE	Email istituzionale
password	varchar(255)	NOT NULL	Hash Argon2 della password

has_2fa	boolean	DEFAULT FALSE	2FA abilitata
metodo_2fa	varchar(50)	—	Metodo del secondo fattore
registration_with_identity_provider	boolean	DEFAULT FALSE	Registrazione via ID Provider
identity_provider	varchar(50)	—	ID Provider usato (SPID/CIE)

5.3.2 token

Attributo	Tipo	Vincoli	Descrizione
id	uuid	PK	Identificativo del token
valore	varchar(255)	NOT NULL, UNIQUE	Valore del token/OTP
scadenza	timestamp	NOT NULL	Data e ora di scadenza
tipo	varchar(50)	NOT NULL	Tipo (registrazione/2FA/recupero)
utente_uuid	varchar(36)	FK (utente_afam), NOT NULL	Riferimento all'account utente
accessTime	timestamp	—	Ultimo timestamp di accesso/utilizzo

5.3.3 profilo

Attributo	Tipo	Vincoli	Descrizione
id	uuid	PK	Identificativo del profilo
utente_uuid	uuid	FK (utente_afam), NOT NULL, UNIQUE	Account associato (Relazione 1:1)
descrizione	text	—	Presentazione dell'utente
policy_visibilita	varchar(50)	—	Visibilità del profilo

foto_profilo	varchar(255)	—	Percorso o URL della foto profilo
nome	varchar(50)	NOT NULL	Nome dell'utente
cognome	varchar(50)	NOT NULL	Cognome dell'utente
data_nascita	date	—	Data di nascita
codice_fiscale	varchar(16)	UNIQUE	Codice fiscale
citta	varchar(100)	—	Città di residenza
indirizzo	varchar(255)	—	Indirizzo di residenza
telefono	varchar(20)	—	Numero di telefono
istituzione	varchar(100)	—	Nome dell'istituto/università
dominio_istituzionale	varchar(100)	—	Dominio email dell'istituzione
matricola	varchar(50)	—	Matricola universitaria/meccanografica
corso_di_studi	varchar(100)	—	Corso di studi frequentato
anno_accademico	varchar(20)	—	Anno accademico corrente

5.3.4 profilo_competenze

Attributo	Tipo	Vincoli	Descrizione
profilo_id	uuid	PK, FK (profilo)	Riferimento al profilo
competenza	varchar(255)	PK	Singola competenza associata al profilo

5.3.5 profilo_interessi

Attributo	Tipo	Vincoli	Descrizione
profilo_id	uuid	PK, FK (profilo)	Riferimento al profilo

interesse	varchar(255)	PK	Singolo interesse associato al profilo
------------------	--------------	----	--

5.3.6 contenuto

Attributo	Tipo	Vincoli	Descrizione
id	uuid	PK	Identificativo del contenuto
titolo	varchar(255)	NOT NULL	Titolo del contenuto inserito
tipo	varchar(50)	NOT NULL	Tipo di contenuto
descrizione	text	—	Descrizione testuale
policy_visibilita	varchar(50)	—	Impostazioni di visibilità
numero_visualizzazioni	int	DEFAULT 0	Conteggio delle visualizzazioni ricevute
profilo_id	uuid	FK (profilo), NOT NULL	Profilo dell'utente creatore/proprietario

5.3.7 contenuto_autore

Attributo	Tipo	Vincoli	Descrizione
contenuto_id	uuid	PK, FK (contenuto)	Riferimento al contenuto
nome_autore	varchar(255)	NOT NULL	Nome dell'autore dell'opera/contenuto

5.3.8 contenuto_collaboratore

Attributo	Tipo	Vincoli	Descrizione
contenuto_id	uuid	PK, FK (contenuto)	Riferimento al contenuto

nome_collaboratore	varchar(255)	NOT NULL	Nome del collaboratore associato
---------------------------	--------------	----------	----------------------------------

5.3.9 allegato

Attributo	Tipo	Vincoli	Descrizione
id	uuid	PK	Identificativo dell'allegato
contenuto_id	uuid	FK (contenuto), NOT NULL	Contenuto a cui l'allegato è associato
url_file	varchar(255)	NOT NULL	Percorso o URL del file fisico memorizzato

5.3.10 gruppo

Attributo	Tipo	Vincoli	Descrizione
id	uuid	PK	Identificativo del gruppo
nome	varchar(100)	NOT NULL	Nome assegnato al gruppo
profilo_id	uuid	FK (profilo), NOT NULL	Profilo dell'utente amministratore/ creatore

5.3.11 link

Attributo	Tipo	Vincoli	Descrizione
identificatore_link	varchar(255)	PK	Stringa/Hash univoco del link di condivisione
data_scadenza	timestamp	—	Data e ora di fine validità del link
risorsa_associata	varchar(255)	—	Tipo o percorso della risorsa condivisa

stato	varchar(50)	—	Stato del link (es. Attivo, Scaduto, Disattivato)
impostazioni	text	—	Configurazioni o restrizioni aggiuntive in formato testo/JSON
numero_visualizzazioni	int	DEFAULT 0	Conteggio di quante volte il link è stato aperto
profilo_id	uuid	FK (profilo)	Profilo associato al link (opzionale)
gruppo_id	uuid	FK (gruppo)	Gruppo associato al link (opzionale)
contenuto_id	uuid	FK (contenuto)	Contenuto associato al link (opzionale)

5.3.12 gruppo_contenuto

Attributo	Tipo	Vincoli	Descrizione
gruppo_id	uuid	PK, FK (gruppo)	Riferimento al gruppo
contenuto_id	uuid	PK, FK (contenuto)	Riferimento al contenuto inserito nel gruppo

6. Controllo degli accessi e sicurezza

L'accesso al sistema è regolato da Spring Security con politica di sessione stateless. Dopo un'autenticazione riuscita (e, ove attiva, la verifica del secondo fattore OTP), il server emette un token JWT che il client trasmette nell'header Authorization a ogni richiesta. Un filtro di autenticazione (JwtAuthenticationFilter) valida il token e popola il contesto di sicurezza. Il sistema distingue due ruoli: AFAM, assegnato agli utenti autenticati con pieno accesso alle funzionalità del proprio profilo e contenuti, e GUEST, assegnato ai Visualizzatori tramite un token ospite, con accesso limitato alla ricerca e alla visualizzazione dei contenuti pubblici. Le rotte di autenticazione, registrazione, recupero e i link di condivisione pubblici sono accessibili senza autenticazione; tutte le altre richiedono il ruolo AFAM. Le password sono cifrate con Argon2 e mai memorizzate in chiaro; gli input sono validati e sanificati; la fruizione dei link rispetta stato e scadenza, restituendo gli opportuni codici HTTP (es. 410 Gone per link scaduti o revocati).

7. Condizioni limite

Avvio: l'infrastruttura è avviata tramite docker-compose, che istanzia in sequenza il nodo Database, il nodo Backend (che valida lo schema all'avvio) e il nodo Frontend.

Arresto: l'arresto controllato dei container preserva i dati persistiti su volume dedicato del DBMS.

Gestione dei guasti: in caso di perdita di connessione o di assenza di risposta del DBMS, il sistema gestisce l'errore in modo controllato — riportando l'utente a uno stato valido, eliminando i dati non ancora inviati e mostrando un messaggio dedicato — e adotta una strategia di ritentativi con attesa crescente prima di porre in stato di sospensione il solo nodo interessato, in attesa di intervento manutentivo, coerentemente con i casi d'uso di comportamento eccezionale del RAD.

8. Considerazioni aggiuntive

Poichè si è scelto di adottare un'architettura in microservizi (mediante docker), si reputa consono, al fine della fase di produzione di tale software, fornire alcune considerazioni circa la struttura dei microservizi.

Potrebbe essere consono aggiungere due diversi container: uno che memorizza in maniera persistente i dati (contenuti) degli utenti, tale container risulterebbe isolato ed inaccessibile dall'esterno; un altro che provvede a svolgere il compito di gateway, accettando, validando e smistando le richieste provenienti dall'esterno della rete, verso gli adeguati servizi.

Se si volesse adottare una struttura ancor più precisa, potrebbe risultare necessario suddividere il servizio di backend in due diversi servizi: uno ad uso esclusivo dell'utente AFAM, l'altro ad uso esclusivo del visualizzatore. Questo incrementerebbe ulteriormente la sicurezza, mitigando attacchi volti al sistema da parte di utenti non loggati.

Infine, potrebbe risultare necessario introdurre un servizio ad'hoc circa la telemetria e l'analisi dei log del server (inteso come insieme dei microservizi). A tal fine si evidenzia la necessità di costruire un'infrastruttura di rete (software –tramite docker e programmi dedicati-) robusta e resiliente, volta a mitigare quanto più possibile il rischio di fuga di dati, per come imposto da: **GDPR** (Regolamento UE 2016/679) e **Diritto d'Autore** (Legge 633/1941).